

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE CODE: ITA 0606

COURSE NAME: MACHINE LEARNING

COURSE OUTCOME COVERED

CO5: *Introduction – NumPy Arrays – NumPy Basics- Math – Random – Indexing – Filtering – Statistics – Aggregation – saving data – Data analysis with Pandas – DataFrame – Combining – Indexing – File I/O – Grouping – Filtering – Sorting – Plotting*
(BL 3)

Assessment Tool Weightage: 20%

QUESTIONS:15

Total Marks:25

Annexure A

Pseudocode Writing Task (Algorithm Design)

Code of Conduct:

*I ADHITHYA.S 192412352 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:

Annexure A

1. Design a pseudocode algorithm to read a dataset from a file and preprocess it using array and DataFrame operations. The algorithm should handle missing values, remove duplicate records, normalize numerical columns, and standardize categorical values. Include steps for indexing, filtering invalid entries, and saving the cleaned dataset. Ensure the algorithm clearly defines input, processing steps, and output while maintaining computational efficiency.
2. Write a detailed pseudocode algorithm to compute statistical measures such as mean, median, standard deviation, and variance from a large dataset. The algorithm should demonstrate how to efficiently use array-based operations, apply filtering to exclude unwanted values, and perform aggregation. Include steps for validating the dataset and handling edge cases such as empty arrays or extreme outliers.
3. Develop a pseudocode algorithm to combine two or more DataFrames containing related data. The algorithm must include steps for aligning indices, merging datasets based on a common key, removing duplicates, and validating the integrity of the merged data. Incorporate grouping and sorting operations, and ensure that constraints such as consistency and completeness of records are maintained.
4. Create a pseudocode algorithm that filters a dataset based on multiple conditions (e.g., value ranges, categories) and then sorts the filtered results. The algorithm should utilize indexing techniques, logical conditions, and efficient sorting methods. Include steps for handling large datasets and optimizing performance while ensuring that the output meets all filtering constraints.
5. Write a pseudocode algorithm for a complete data analysis workflow that includes reading data from files, converting it into DataFrames, performing grouping and aggregation, and generating plots. The algorithm should specify steps for handling file input/output errors, applying transformations, and producing summarized results suitable for visualization. Ensure clarity in each stage from data loading to final output generation.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Conceptual Understanding	25	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	25	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	20	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	15	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	15	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand

CO5 AT2 - Pseudocode Writing Task (Algorithm Design)

Name: ADHITHYA.S

Register Number: 192412352

Course Code: ITA0606

Course Name: MACHINE LEARNING

Task 1 — Data Loading and Preprocessing Algorithm

This algorithm reads a raw dataset from file, then cleans and standardizes it using array and DataFrame operations. It addresses missing values, duplicate records, numerical normalization, categorical standardization, indexing, and validity filtering, before persisting the cleaned dataset.

INPUT

- file_path: path to the raw dataset file (CSV/Excel/JSON)
- num_columns: list of numerical column names to normalize
- cat_columns: list of categorical column names to standardize
- output_path: destination path for the cleaned dataset

OUTPUT

- cleaned_df: a validated, deduplicated, normalized, indexed DataFrame written to output_path

ALGORITHM

```
ALGORITHM PreprocessDataset(file_path, num_columns, cat_columns, output_path)

// ----- Step 1: Load data -----
IF NOT FileExists(file_path) THEN
  RAISE Error("File not found: " + file_path)
END IF
TRY
  raw_df <- ReadFileToDataFrame(file_path) // infers CSV/JSON/Excel by extension
CATCH ParseError AS e
  RAISE Error("Failed to parse file: " + e.message)
END TRY
IF raw_df.RowCount = 0 THEN
  RAISE Error("Dataset is empty")
END IF

// ----- Step 2: Establish index -----
IF "id" IN raw_df.Columns THEN
  raw_df.SetIndex("id") // use natural key when available
ELSE
  raw_df.SetIndex(RangeIndex(0, raw_df.RowCount)) // fallback surrogate index
END IF
```

```

// ----- Step 3: Handle missing values -----
FOR EACH col IN raw_df.Columns DO
  missing_ratio <- CountNulls(raw_df[col]) / raw_df.RowCount
  IF missing_ratio > 0.5 THEN
    raw_df.DropColumn(col)    // too sparse to be useful
  ELSE IF col IN num_columns THEN
    median_val <- Median(raw_df[col] WHERE NOT IsNull())
    raw_df[col].FillNull(median_val)
  ELSE IF col IN cat_columns THEN
    mode_val <- Mode(raw_df[col] WHERE NOT IsNull())
    raw_df[col].FillNull(mode_val)
  END IF
END FOR

// ----- Step 4: Remove duplicate records -----
raw_df <- raw_df.DropDuplicates(subset = ALL_COLUMNS, keep = "first")

// ----- Step 5: Filter invalid entries -----
FOR EACH col IN num_columns DO
  raw_df <- raw_df.Filter(IsFinite(raw_df[col]) AND raw_df[col] IS NOT NULL)
END FOR
raw_df <- raw_df.Filter(RowHasNoNullInRequiredFields(raw_df))

// ----- Step 6: Normalize numerical columns (vectorized) -----
FOR EACH col IN num_columns DO
  col_array <- raw_df[col].ToArray()
  min_v <- Min(col_array)
  max_v <- Max(col_array)
  IF max_v = min_v THEN
    raw_df[col] <- 0.0          // avoid divide-by-zero, constant column
  ELSE
    raw_df[col] <- (col_array - min_v) / (max_v - min_v) // Min-Max scaling
  END IF
END FOR

// ----- Step 7: Standardize categorical values -----
FOR EACH col IN cat_columns DO
  raw_df[col] <- Trim(Lower(raw_df[col])) // normalize case/whitespace
  raw_df[col] <- MapKnownAliases(raw_df[col], ALIAS_TABLE[col])
END FOR

// ----- Step 8: Final structural validation -----
ASSERT raw_df.HasNoDuplicateIndex()
ASSERT raw_df.RowCount > 0

cleaned_df <- raw_df

// ----- Step 9: Persist output -----
TRY
  WriteDataFrameToFile(cleaned_df, output_path)
CATCH IOError AS e
  RAISE Error("Failed to write output: " + e.message)
END TRY

```

```
RETURN cleaned_df  
END ALGORITHM
```

EFFICIENCYNOTES

- Missing-value and normalization passes use vectorized column operations ($O(n)$ per column) rather than per-cell loops.
- DropDuplicates and filtering use set/hash-based comparisons, avoiding $O(n^2)$ pairwise checks.
- Sparse columns are dropped early (Step 3) to reduce the working set before costlier operations.

Task 2 — Statistical Measures Computation Algorithm

This algorithm computes mean, median, standard deviation, and variance from a large dataset column, using vectorized array operations, filtering of unwanted values, and safe handling of edge cases such as empty arrays or extreme outliers.

INPUT

- data_array: array/column of numeric values (potentially very large)
- outlier_method: strategy for outlier exclusion, e.g. "iqr" or "zscore"
- outlier_threshold: numeric threshold associated with the chosen method

OUTPUT

- stats_result: record containing {count, mean, median, std_dev, variance, excluded_count}

ALGORITHM

```
ALGORITHM ComputeStatistics(data_array, outlier_method, outlier_threshold)

// ----- Step 1: Validate input -----
IF data_array IS NULL THEN
  RAISE Error("data_array must not be null")
END IF
n_raw <- Length(data_array)
IF n_raw = 0 THEN
  RETURN { count: 0, mean: NULL, median: NULL, std_dev: NULL,
          variance: NULL, excluded_count: 0, note: "empty input" }
END IF

// ----- Step 2: Filter unwanted values (vectorized) -----
clean_array <- data_array WHERE IsFinite(data_array) AND NOT IsNull(data_array)
excluded_missing <- n_raw - Length(clean_array)

IF Length(clean_array) = 0 THEN
  RETURN { count: 0, mean: NULL, median: NULL, std_dev: NULL,
          variance: NULL, excluded_count: excluded_missing, note: "no valid values" }
END IF

// ----- Step 3: Exclude extreme outliers -----
IF outlier_method = "iqr" THEN
  q1 <- Percentile(clean_array, 25)
  q3 <- Percentile(clean_array, 75)
  iqr <- q3 - q1
  lower_bound <- q1 - outlier_threshold * iqr
  upper_bound <- q3 + outlier_threshold * iqr
  filtered_array <- clean_array WHERE clean_array >= lower_bound
                          AND clean_array <= upper_bound
ELSE IF outlier_method = "zscore" THEN
  mu_temp <- Mean(clean_array)
  sigma_temp <- StdDev(clean_array)
  IF sigma_temp = 0 THEN
    filtered_array <- clean_array    // no spread; nothing to exclude
  ELSE
    z_scores <- (clean_array - mu_temp) / sigma_temp
```

```

        filtered_array <- clean_array WHERE Abs(z_scores) <= outlier_threshold
    END IF
ELSE
    filtered_array <- clean_array          // no outlier filtering requested
END IF

excluded_outliers <- Length(clean_array) - Length(filtered_array)
n <- Length(filtered_array)

IF n = 0 THEN
    RETURN { count: 0, mean: NULL, median: NULL, std_dev: NULL, variance: NULL,
            excluded_count: excluded_missing + excluded_outliers,
            note: "all values excluded as outliers" }
END IF

// ----- Step 4: Aggregate statistics (single-pass where possible) -----
total <- Sum(filtered_array)              // vectorized reduction
mean_val <- total / n

sorted_array <- Sort(filtered_array)      // O(n log n)
IF n MOD 2 = 1 THEN
    median_val <- sorted_array[(n - 1) / 2]
ELSE
    mid_r <- n / 2
    median_val <- (sorted_array[mid_r - 1] + sorted_array[mid_r]) / 2
END IF

IF n > 1 THEN
    squared_diffs <- (filtered_array - mean_val) ^ 2 // vectorized
    variance_val <- Sum(squared_diffs) / (n - 1)    // sample variance
    std_dev_val <- SquareRoot(variance_val)
ELSE
    variance_val <- 0.0
    std_dev_val <- 0.0
END IF

// ----- Step 5: Assemble result -----
stats_result <- {
    count: n,
    mean: mean_val,
    median: median_val,
    std_dev: std_dev_val,
    variance: variance_val,
    excluded_count: excluded_missing + excluded_outliers
}

RETURN stats_result
END ALGORITHM

```

EFFICIENCYNOTES

- Filtering, sum, and squared-difference steps use vectorized array operations ($O(n)$) instead of scalar loops.
- Sorting (required only for median) is the single $O(n \log n)$ step; all other statistics remain $O(n)$.
- Edge cases (empty array, all-null, all-outlier) are checked before expensive aggregation to fail fast.

Task 3 — DataFrame Merging Algorithm

This algorithm merges two or more related DataFrames on a common key, aligning indices, removing duplicates, validating referential integrity, and applying grouping and sorting to the combined result.

INPUT

- `df_list`: list of DataFrames to merge (`df1`, `df2`, ..., `dfk`)
- `merge_key`: column name common to all DataFrames used as the join key
- `join_type`: "inner", "left", "right", or "outer"
- `group_by_column`: optional column for post-merge aggregation
- `sort_column`: column to sort the final result by

OUTPUT

- `merged_df`: a validated, deduplicated, sorted DataFrame combining all inputs

ALGORITHM

```
ALGORITHM MergeDataFrames(df_list, merge_key, join_type, group_by_column, sort_column)
```

```
// ----- Step 1: Validate inputs -----
```

```
IF Length(df_list) < 2 THEN
```

```
  RAISE Error("At least two DataFrames are required")
```

```
END IF
```

```
FOR EACH df IN df_list DO
```

```
  IF merge_key NOT IN df.Columns THEN
```

```
    RAISE Error("Missing merge key in one of the DataFrames")
```

```
  END IF
```

```
END FOR
```

```
// ----- Step 2: Align indices -----
```

```
FOR EACH df IN df_list DO
```

```
  df.SetIndex(merge_key)      // align on the common key for fast lookups
```

```
  df.SortIndex()              // ensures consistent ordering pre-merge
```

```
END FOR
```

```
// ----- Step 3: Iteratively merge -----
```

```
merged_df <- df_list[0]
```

```
FOR i FROM 1 TO Length(df_list) - 1 DO
```

```
  next_df <- df_list[i]
```

```
  merged_df <- Merge(merged_df, next_df,
```

```
    on = merge_key,
```

```
    how = join_type,
```

```
    suffixes = ("_L", "_R"))
```

```
END FOR
```

```
// ----- Step 4: Remove duplicate records -----
```

```
merged_df <- merged_df.DropDuplicates(subset = ALL_COLUMNS, keep = "first")
```

```
merged_df <- merged_df.Filter(NOT merged_df.Index.HasDuplicates())
```

```
// ----- Step 5: Validate integrity -----
```

```
FOR EACH col IN merged_df.Columns DO
```

```
  IF IsRequiredColumn(col) AND CountNulls(merged_df[col]) > 0 THEN
```

```

    LogWarning("Nulls found in required column: " + col)
    merged_df <- merged_df.Filter(NOT IsNull(merged_df[col]))
  END IF
END FOR
orphan_count <- CountRowsWithNoMatch(merged_df, df_list, merge_key)
IF orphan_count > 0 AND join_type = "inner" THEN
  RAISE Error("Unexpected unmatched rows after inner join")
END IF

// ----- Step 6: Grouping (optional) -----
IF group_by_column IS NOT NULL THEN
  grouped <- merged_df.GroupBy(group_by_column)
  merged_df <- grouped.Aggregate({
    numeric_columns: "mean",
    count_columns: "count"
  })
END IF

// ----- Step 7: Sorting -----
IF sort_column IS NOT NULL AND sort_column IN merged_df.Columns THEN
  merged_df <- merged_df.SortValues(by = sort_column, ascending = TRUE)
END IF

// ----- Step 8: Final consistency checks -----
ASSERT merged_df.HasNoDuplicateIndex()
ASSERT merged_df.RowCount > 0

RETURN merged_df
END ALGORITHM

```

EFFICIENCYNOTES

- Setting and sorting the join key as an index enables hash/merge-join strategies rather than nested-loop comparisons.
- Duplicate removal and null filtering operate on vectorized boolean masks.
- GroupBy aggregation is applied only once, after merging, to avoid repeated grouping passes on intermediate DataFrames.

Task 4 — Multi-Condition Filtering and Sorting Algorithm

This algorithm filters a dataset using multiple simultaneous conditions (value ranges and categories) and then sorts the results efficiently, with attention to performance on large datasets.

INPUT

- df: input DataFrame
- range_conditions: list of tuples (column, min_value, max_value)
- category_conditions: list of tuples (column, allowed_values_set)
- sort_columns: ordered list of (column, direction) pairs for multi-key sort

OUTPUT

- result_df: filtered and sorted DataFrame meeting all constraints

ALGORITHM

```
ALGORITHM FilterAndSort(df, range_conditions, category_conditions, sort_columns)
```

```
// ----- Step 1: Validate inputs -----
IF df.RowCount = 0 THEN
    RETURN df          // nothing to filter
END IF
FOR EACH (col, _, _) IN range_conditions DO
    IF col NOT IN df.Columns THEN
        RAISE Error("Unknown range column: " + col)
    END IF
END FOR
FOR EACH (col, _) IN category_conditions DO
    IF col NOT IN df.Columns THEN
        RAISE Error("Unknown category column: " + col)
    END IF
END FOR

// ----- Step 2: Build a combined boolean mask (vectorized) -----
mask <- Array(TRUE, length = df.RowCount) // start with all rows included

FOR EACH (col, min_v, max_v) IN range_conditions DO
    mask <- mask AND (df[col] >= min_v) AND (df[col] <= max_v)
END FOR

FOR EACH (col, allowed_values) IN category_conditions DO
    allowed_set <- ToHashSet(allowed_values) // O(1) membership checks
    mask <- mask AND df[col].IsIn(allowed_set)
END FOR

// ----- Step 3: Apply filter using indexing -----
filtered_df <- df[mask] // boolean indexing, single pass

IF filtered_df.RowCount = 0 THEN
    RETURN filtered_df // no matches; return empty result early
END IF
```

```

// ----- Step 4: Sort efficiently on multiple keys -----
// Use a stable sort so equal keys preserve prior relative order
result_df <- filtered_df.SortValues(
  by = ExtractColumns(sort_columns),
  ascending = ExtractDirections(sort_columns),
  algorithm = "stable"    // e.g., timsort, O(n log n)
)

// ----- Step 5: Reset index for downstream use -----
result_df <- result_df.ResetIndex(drop = TRUE)

// ----- Step 6: Verify constraints -----
FOR EACH (col, min_v, max_v) IN range_conditions DO
  ASSERT ALL(result_df[col] >= min_v AND result_df[col] <= max_v)
END FOR
FOR EACH (col, allowed_values) IN category_conditions DO
  ASSERT ALL(result_df[col].IsIn(allowed_values))
END FOR

RETURN result_df
END ALGORITHM

```

EFFICIENCY/LARGE-DATASET NOTES

- All conditions are combined into a single boolean mask before any row copy is made, avoiding repeated intermediate DataFrame allocations.
- Category membership tests use hash sets for O(1) lookups instead of O(k) linear scans per row.
- An early-exit check after filtering avoids an unnecessary sort when the result is empty.
- For very large datasets, the mask can be computed in chunks/partitions and filtering applied per chunk to bound peak memory usage.

Task 5 — End-to-End Data Analysis Workflow Algorithm

This algorithm defines a complete workflow: reading data from files, converting to DataFrames, applying transformations, performing grouping and aggregation, and producing summarized results suitable for visualization, with explicit error handling at each I/O boundary.

INPUT

- `file_paths`: list of file paths to load (one or more sources)
- `transform_steps`: ordered list of transformation functions to apply
- `group_by_column`: column used for grouping
- `agg_spec`: mapping of column -> aggregation function (e.g., "sum", "mean")
- `plot_spec`: description of desired chart type and axes

OUTPUT

- `summary_df`: aggregated result table
- `chart_object`: generated plot ready for display/export

ALGORITHM

```
ALGORITHM RunAnalysisWorkflow(file_paths, transform_steps, group_by_column, agg_spec, plot_spec)
```

```
// ----- Step 1: Read data from file(s) with error handling -----
raw_frames <- EmptyList()
FOR EACH path IN file_paths DO
  TRY
    IF NOT FileExists(path) THEN
      RAISE FileNotFoundError(path)
    END IF
    frame <- ReadFileToDataFrame(path)    // handles CSV/JSON/Excel by extension
    Append(raw_frames, frame)
  CATCH FileNotFoundError AS e
    LogError("Missing file: " + e.path)
    CONTINUE    // skip and proceed with remaining files
  CATCH ParseError AS e
    LogError("Could not parse " + path + ": " + e.message)
    CONTINUE
END TRY
END FOR

IF Length(raw_frames) = 0 THEN
  RAISE Error("No valid input files could be loaded")
END IF

// ----- Step 2: Combine into a single working DataFrame -----
IF Length(raw_frames) = 1 THEN
  df <- raw_frames[0]
ELSE
  df <- ConcatDataFrames(raw_frames, axis = "rows", ignore_index = TRUE)
END IF

// ----- Step 3: Apply transformations in sequence -----
FOR EACH transform_fn IN transform_steps DO
```

```

TRY
  df<- transform_fn(df)           // e.g., type casting, scaling, derived cols
CATCH TransformError AS e
  LogError("Transform failed: " + e.message)
  RAISE Error("Workflow halted at transformation step")
END TRY
END FOR

// ----- Step 4: Basic cleaning before aggregation -----
df<- df.DropDuplicates()
df<- df.Filter(NOT IsNull(df[group_by_column]))

IF df.RowCount = 0 THEN
  RAISE Error("No rows remain after cleaning; cannot aggregate")
END IF

// ----- Step 5: Group and aggregate -----
grouped<- df.GroupBy(group_by_column)
summary_df<- grouped.Aggregate(agg_spec)
summary_df<- summary_df.SortValues(by = group_by_column, ascending = TRUE)
summary_df<- summary_df.ResetIndex(drop = TRUE)

// ----- Step 6: Generate visualization -----
TRY
  chart_object<- CreatePlot(
    data = summary_df,
    type = plot_spec.chart_type, // e.g., "bar", "line", "scatter"
    x = plot_spec.x_axis,
    y = plot_spec.y_axis,
    title = plot_spec.title
  )
CATCH PlottingError AS e
  LogError("Plot generation failed: " + e.message)
  chart_object<- NULL // still return numeric summary
END TRY

// ----- Step 7: Persist outputs -----
TRY
  WriteDataFrameToFile(summary_df, "summary_output.csv")
  IF chart_object IS NOT NULL THEN
    SaveChartToFile(chart_object, "summary_chart.png")
  END IF
CATCH IOError AS e
  LogError("Failed to save outputs: " + e.message)
END TRY

RETURN (summary_df, chart_object)
END ALGORITHM

```

WORKFLOW NOTES

- File-read errors are caught per file so one bad source does not abort the entire batch.
- Each pipeline stage (read → combine → transform → clean → aggregate → plot → save) is isolated with its own error handling for easier debugging and partial-failure recovery.

- Grouping and aggregation are performed once on the fully-transformed DataFrame to avoid redundant computation.
- Plot generation failures degrade gracefully: the numeric summary is still returned and saved even if visualization fails.